Conceptos de Sistemas Operativos (IC)

Introducción a los Sistemas Operativos (LI, LS, APU, ATIC)

Trabajo Práctico N° 1

Objetivo

El objetivo de esta práctica es que el estudiante se familiarice con los conceptos básicos del sistema operativo *GNU/Linux*, su instalación, entorno y comandos principales. Manejo de Usuarios, permisos y su sistemas de archivos.

Temas Incluidos

GNU/Linux, instalación y conceptos básicos, permisos, arranque, usuarios. organización interna.

1. Características de *GNU/Linux*:
   a. Mencione y explique las características más relevantes de *GNU/Linux*.

   Libre y de Código Abierto (Open Source): Se distribuye bajo la licencia GPL (General Public License). Esto significa que los usuarios tienen la libertad de ejecutar, copiar, distribuir, estudiar, cambiar y mejorar el software.

   Multiusuario: Permite que varios usuarios accedan a los recursos del sistema (memoria, programas, archivos) simultáneamente, protegiendo los datos de cada uno.

   Multitarea: Capacidad de ejecutar múltiples procesos o programas al mismo tiempo sin interferir entre sí.

   Multiplataforma (Portabilidad): Puede funcionar en una gran variedad de arquitecturas de hardware, desde supercomputadoras y servidores hasta relojes inteligentes y routers.

   Estabilidad y Seguridad: Es conocido por su robustez; es difícil que el sistema se "cuelgue". Además, su gestión de permisos y usuarios lo hace muy seguro contra virus y malware en comparación con otros sistemas.

   b. Mencione otros sistemas operativos y compárelos con *GNU/Linux* en cuanto a los puntos mencionados en el inciso *a*.

Característica	GNU/Linux	Microsoft Windows	macOS
Licencia	GPL (Libre y gratuito mayormente)	Propietario (Licencia de pago)	Propietario (Vinculado al HW Apple)

Característica	GNU/Linux	Microsoft Windows	macOS
Código	Abierto (Visible y modificable)	Cerrado (Secreto comercial)	Híbrido (Núcleo Darwin abierto, GUI cerrada)
Multiusuario	Nativo y robusto (herencia Unix)	Soportado, pero enfocado históricamente al usuario único/escritorio	Nativo (herencia Unix/BSD)
Estabilidad	Alta (Ideal para servidores)	Media/Alta (Ha mejorado, pero requiere reinicios)	Alta
Personalización	Total (Kernel, GUI, todo es modificable)	Limitada (Apariencia y configuraciones básicas)	Baja (Apple define la experiencia)

c. ¿Qué es **GNU**?

GNU es un acrónimo recursivo que significa **"GNU's Not Unix"** (GNU No es Unix). Es un proyecto iniciado para crear un sistema operativo completo que fuera totalmente software libre y compatible con Unix. El proyecto GNU proporcionó muchas de las herramientas esenciales que utiliza el sistema hoy en día (compiladores, editores de texto, intérpretes de comandos o *shells*, bibliotecas), pero le faltaba el núcleo (kernel).

d. Indique una breve historia sobre la evolución del proyecto *GNU*.

1983: Richard Stallman anuncia el proyecto GNU con el objetivo de crear un sistema operativo libre tipo Unix.

1985: Se funda la Free Software Foundation (FSF) y se crea la licencia GPL para garantizar legalmente las libertades del software.

Años 80: Se desarrollan herramientas clave como el compilador GCC, el editor Emacs y la shell Bash. Sin embargo, el desarrollo de su propio núcleo, llamado GNU Hurd, se retrasa por problemas técnicos y de diseño.

1991: Linus Torvalds, un estudiante finlandés, crea un núcleo (kernel) llamado Linux como proyecto personal.

1992: Linus libera el kernel Linux bajo la licencia GPL. La comunidad integra el kernel Linux con las herramientas GNU ya existentes, dando nacimiento al sistema operativo completo que hoy llamamos GNU/Linux.

e. Explique qué es la multitarea, e indique si *GNU/Linux* hace uso de ella.

Sí. GNU/Linux utiliza **multitarea apropiativa (preemptive multitasking)**. Esto significa que el sistema operativo (el kernel) tiene el control total para pausar un proceso y dar paso a otro, asegurando que ningún programa bloquee la CPU indefinidamente.

f. ¿Qué es **POSIX**?

POSIX (Portable Operating System Interface for uniX) es un conjunto de estándares definidos por la IEEE. Su objetivo es mantener la compatibilidad entre diferentes sistemas operativos variantes de Unix. Define cómo deben comportarse las interfaces de programación (API), las shells y las utilidades. Esto permite que un programa escrito para un sistema compatible con POSIX pueda compilarse y ejecutarse en otro sistema POSIX con cambios mínimos. GNU/Linux sigue mayoritariamente los estándares POSIX.

2. Distribuciones de *GNU/Linux*:

a. ¿Qué es una distribución de *GNU/Linux*? Nombre al menos 4 distribuciones de *GNU/Linux* y cite diferencias básicas entre ellas.

Una **distribución** (o "distro") es un sistema operativo completo y funcional que se construye empaquetando el núcleo **Linux** junto con las herramientas del proyecto **GNU**, un sistema de gestión de paquetes, un entorno de escritorio (interfaz gráfica), controladores y software adicional preinstalado. Como el Kernel Linux es de código abierto, cualquiera puede tomarlo, agregarle programas y crear su propia versión o "sabor" del sistema.

Debian:

Enfoque: Estabilidad extrema y filosofía de Software Libre puro.

Diferencia: Es la base de muchas otras (como Ubuntu). Su ciclo de actualizaciones es lento porque priorizan que nada falle antes que tener la "última versión". Usa paquetería .deb y apt.

Ubuntu:

Enfoque: Facilidad de uso para el usuario final y escritorio.

Diferencia: Basada en Debian pero patrocinada por una empresa (Canonical). Incluye drivers privativos (para Wi-Fi o tarjetas de video) más fácilmente para que todo funcione "out of the box".

Fedora:

Enfoque: Innovación y nuevas tecnologías.

Diferencia: Es el campo de pruebas de Red Hat (RHEL). Siempre trae lo último de lo último (versiones de Kernel y GNOME más nuevas). Usa paquetería .rpm y dnf.

Arch Linux:

Enfoque: Minimalismo, control total y modelo "Rolling Release".

Diferencia: No se instala un sistema completo de golpe; tú lo construyes pieza por pieza. No tiene "versiones" (ej. versión 20, 21), sino que se actualiza continuamente día a día.

b. ¿En qué se diferencia una distribución de otra?
Aunque todas usan el mismo núcleo (Linux), se diferencian principalmente en:

Gestor de Paquetes: La herramienta para instalar programas (ej: apt en Debian/Ubuntu vs dnf en Fedora vs pacman en Arch).

Ciclo de Lanzamiento:

Stable/LTS: Versiones fijas cada X años (ej. Debian, Ubuntu LTS). Priorizan estabilidad.

Rolling Release: Actualización constante sin versiones mayores (ej. Arch Linux). Priorizan novedad.

Entorno de Escritorio por defecto: La interfaz visual que te presentan (GNOME, KDE Plasma, XFCE, etc.).

Filosofía: Algunas son 100% comunitarias (Debian) y otras comerciales (RHEL, Ubuntu). Algunas aceptan software propietario (drivers de Nvidia) y otras lo prohíben estrictamente.

c. ¿Qué es **Debian**? Acceda al sitio https://www.debian.org/ e indique cuáles son los objetivos del proyecto y una breve cronología del mismo.
Debian es una de las distribuciones más antiguas y respetadas. Es un sistema operativo universal desarrollado íntegramente por una comunidad de voluntarios (no hay una empresa dueña detrás) bajo el Proyecto Debian.

Objetivos (El Contrato Social de Debian):

100% Software Libre: Debian se compromete a permanecer completamente libre.

Retribuir a la comunidad: Las mejoras que hacen se envían a los autores originales.

No ocultar problemas: Mantener la base de datos de errores abierta al público.

Prioridad: Los usuarios y el software libre son lo primero.

Breve Cronología:

1993: Fundado por Ian Murdock (de ahí el nombre Debra + Ian). Lanzó el "Manifiesto Debian" pidiendo una distribución abierta y mantenida por la comunidad, al estilo de Linux y GNU.

1996 (Versión 1.1 "Buzz"): Primera versión estable con nombres en código basados en los personajes de la película Toy Story (tradición que se mantiene hasta hoy; la versión inestable siempre se llama "Sid", el niño que rompe los juguetes).

Actualidad: Se ha convertido en la distribución "madre" de la que derivan cientos de otras distros, incluyendo la popular Ubuntu.

3. Estructura de *GNU/Linux*:

a. Nombre cuáles son los 3 componentes fundamentales de *GNU/Linux*.

El Kernel (Núcleo): Es el corazón del sistema operativo. Administra el hardware (memoria, CPU, discos) y proporciona servicios básicos a los programas.

El Shell (Intérprete de Comandos): Es la interfaz principal entre el usuario y el Kernel. Recibe tus órdenes y se las traduce al núcleo para que sean ejecutadas.

El Sistema de Archivos (File System): Es la estructura lógica que organiza cómo se almacenan, nombran y recuperan los datos en el disco. En Linux, "todo es un archivo".

b. Mencione y explique la estructura básica del Sistema Operativo *GNU/Linux*.

La estructura de GNU/Linux se suele representar como un modelo de capas de cebolla, donde cada capa aísla y protege a la interior. De adentro hacia afuera, la estructura es:

Hardware (La capa física): Son los componentes tangibles de la computadora: procesador, memoria RAM, disco duro, placa de red, etc. El software necesita interactuar con ellos, pero no lo hace directamente.

El Kernel (Modo Privilegiado): Envuelve al hardware. Es el único programa que tiene acceso directo y total a los componentes físicos. Actúa como intermediario ("driver"), decidiendo qué programa usa qué recurso y por cuánto tiempo. Si una aplicación intenta tocar la memoria de otra o acceder al hardware directamente sin permiso, el Kernel la detiene para evitar que el sistema falle (Panic).

Llamadas al Sistema (System Calls): Es la frontera o interfaz entre el Kernel y el resto del sistema. Son la forma en que los programas le piden "favores" al Kernel (como "lee este archivo" o "abre una conexión de red").

Espacio de Usuario (User Space): Es la capa externa donde "viven" los usuarios y los programas normales. Aquí se encuentran:

Librerías del Sistema: Código común que usan los programas para funcionar.

Shell: La línea de comandos donde el usuario da instrucciones.

Aplicaciones: Navegadores, editores de texto, servidores web, compiladores, etc.

4. Kernel:
   a. ¿Cuáles son sus funciones principales?
   El Kernel es el núcleo del sistema operativo y actúa como intermediario entre el software (tus programas) y el hardware (la máquina física). Sus funciones principales son:

   Administración de Memoria: Decide cuánta memoria RAM se asigna a cada programa y gestiona la memoria virtual (Swap) cuando la RAM se llena.

   Gestión de Procesos (Scheduler): Determina qué programa usa la CPU, durante cuánto tiempo y en qué orden (multitarea).

   Gestión de Dispositivos (Drivers): Controla los periféricos (teclado, disco, placa de red) a través de controladores para que los programas puedan usarlos sin conocer los detalles eléctricos del hardware.

   Gestión del Sistema de Archivos: Traduce los datos físicos del disco (bits magnéticos) a archivos y carpetas comprensibles para el usuario.

   Llamadas al Sistema (System Calls): Provee una API para que los programas soliciten servicios (como abrir un archivo o conectarse a internet).
   b. ¿Cuál es la versión actual? ¿Cómo se definía el esquema de versionado del Kernel en versiones anteriores a la 2.4? ¿Qué cambió en el versionado que se impuso a partir de la versión 2.6?

   c. ¿Es posible tener más de un Kernel de *GNU/Linux* instalado en la misma máquina?
   Sí, es totalmente posible y muy común.

   Cuando actualizas el sistema, Linux generalmente no borra el Kernel anterior, sino que instala el nuevo junto al viejo.

   En el menú de arranque (GRUB), puedes seleccionar "Opciones avanzadas" y elegir arrancar con una versión antigua del Kernel si la nueva presenta fallas con tu hardware.

   d. ¿Dónde se encuentra ubicado dentro del File System?
   El archivo del Kernel (generalmente llamado vmlinuz) se encuentra en el directorio /boot.

   Ruta típica: /boot/vmlinuz-6.18.0-generic

   Los módulos del kernel (drivers extra) están en /lib/modules.

5. Intérprete de comandos *(Shell)*:

a. ¿Qué es?

El Shell es un programa informático que provee una interfaz de usuario para acceder a los servicios del sistema operativo. Es la capa que "envuelve" al Kernel (de ahí su nombre, "cáscara" o "concha"). Actúa como un intermediario: recibe órdenes escritas por el usuario (o desde un script), las interpreta y se las pasa al Kernel para que este las ejecute

b. ¿Cuáles son sus funciones?

Ejecución de programas: Lanza aplicaciones y comandos.

Gestión de E/S (Entrada/Salida): Permite redirigir la entrada y salida de datos (usando >, <, |).

Interpretación de comandos: Analiza la sintaxis de lo que escribes y lo traduce a instrucciones.

Lenguaje de programación (Scripting): Posee estructuras de control (if, while, for), variables y funciones para automatizar tareas complejas.

Gestión de entorno: Maneja variables de entorno (como el PATH) que afectan cómo corren los programas.

c. Mencione al menos 3 intérpretes de comandos que posee *GNU/Linux* y compárelos entre ellos.

Bash (Bourne Again Shell):

Descripción: Es el estándar "de facto" en la mayoría de las distribuciones Linux.

Ventaja: Gran compatibilidad, muchísima documentación y scripts existentes.

Sh (Bourne Shell):

Descripción: El original de Unix (o su enlace simbólico moderno, dash en Debian/Ubuntu).

Ventaja: Extremadamente ligero y rápido. Se usa mucho para scripts de arranque del sistema, pero es muy básico para uso interactivo (no tiene autocompletado avanzado).

Zsh (Z Shell):

Descripción: Una evolución de Bash con muchas mejoras de usabilidad.

Ventaja: Autocompletado muy superior, corrección ortográfica de comandos y altísima personalización (famoso por el framework "Oh My Zsh").

Fish (Friendly Interactive Shell):

Descripción: Enfocado en ser amigable desde el primer momento.

Ventaja: Colorea la sintaxis mientras escribes y sugiere comandos basados en tu historial automáticamente.

Desventaja: No es 100% compatible con POSIX (scripts de Bash pueden fallar en Fish).

d. ¿Dónde se ubican (*path*) los comandos propios y externos al Shell?

Comandos Internos (Built-ins): No tienen una ubicación física en el disco como archivo ejecutable; viven dentro de la memoria del propio proceso del Shell.

Ejemplos: cd, pwd, alias, history.

Nota: Si buscas which cd a veces no devuelve ruta o dice "shell built-in".

Comandos Externos: Son archivos ejecutables (binarios o scripts) guardados en directorios del disco. El Shell los busca siguiendo la variable de entorno $PATH.

Ubicaciones típicas: /bin, /usr/bin, /sbin, /usr/sbin.

Ejemplos: ls, mkdir, top.

e. ¿Por qué considera que el Shell no es parte del Kernel de *GNU/Linux*?

Estabilidad: Si el Shell falla o se cuelga (algo común en scripts mal hechos), solo se cierra ese programa de usuario; el Kernel sigue funcionando y el sistema no se cae. Si el Shell fuera parte del Kernel, un error ahí colgaría toda la máquina (Kernel Panic).

Flexibilidad: Al ser un programa separado, el usuario tiene la libertad de elegir o cambiar su Shell (usar Zsh en lugar de Bash) sin tener que recompilar o cambiar el sistema operativo entero.

f. ¿Es posible definir un intérprete de comandos distinto para cada usuario? ¿Desde dónde se define? ¿Cualquier usuario puede realizar dicha tarea?

Sí, es posible. Cada usuario puede tener su Shell preferida.
¿Dónde se define?: En el archivo /etc/passwd, que es la base de datos de usuarios locales. Es el último campo de la línea del usuario (ej: /bin/bash o /bin/zsh).
El usuario Root puede cambiar la Shell de cualquier usuario.
Un usuario normal puede cambiar su propia Shell utilizando el comando chsh (Change Shell), siempre y cuando la nueva Shell esté listada en el archivo de seguridad /etc/shells.

6. El sistema de Archivos *(File System)* en Linux:

a. ¿Qué es?

El sistema de archivos (File System) es el método y la estructura de datos que utiliza el sistema operativo para organizar, almacenar, recuperar y administrar los archivos en un dispositivo de almacenamiento (disco rígido, pendrive, etc.). Sin un sistema de archivos, la información guardada en un medio de almacenamiento sería un gran bloque de datos sin sentido, sin saber dónde empieza o termina cada archivo.

En GNU/Linux, existe un concepto fundamental: "Todo es un archivo". Esto significa que no solo los documentos de texto son archivos, sino también los directorios, los dispositivos de hardware (teclado, disco), los sockets de red y los procesos.

b. ¿Cuál es la estructura básica de los File System en *GNU/Linux*? Mencione los directorios más importantes e indique qué tipo de información se encuentra en ellos. ¿A qué hace referencia la sigla **FHS**?

A diferencia de Windows, que usa letras de unidad (C:\, D:\), Linux utiliza una estructura de árbol jerárquico único invertido. Todo el sistema cuelga de un único punto de partida llamado directorio raíz o root, representado por la barra /.

FHS (Filesystem Hierarchy Standard): Estas siglas refieren al Estándar de Jerarquía del Sistema de Archivos. Es una norma que define qué directorios deben existir en un sistema Linux y qué tipo de archivos deben guardarse en cada uno, para garantizar la compatibilidad entre diferentes distribuciones.

Directorio	Descripción del contenido
/ (Root)	Es la raíz de todo el sistema.
/bin	Comandos binarios **esenciales** para todos los usuarios (ej: ls, cp, cat).
/boot	Archivos necesarios para el **arranque** del sistema (aquí vive el Kernel y GRUB).
/dev	Archivos de **dispositivos** (hardware). Ej: /dev/sda es el disco rígido.
/etc	Archivos de **configuración** del sistema. No contiene binarios, solo texto (ej: passwd, configuración de red).
/home	Directorios personales de los **usuarios** (donde guardas tus documentos, descargas, etc.).
/lib	**Librerías** esenciales compartidas para los binarios de /bin y /sbin.
/mnt	Punto de montaje temporal para sistemas de archivos (ej: montar un pendrive manualmente).
/opt	Software **opcional** de terceros que no sigue la estructura estándar (ej: Google Chrome, Zoom).
/proc	Sistema de archivos virtual que documenta el estado del **kernel** y los procesos en memoria (no ocupa espacio en disco).
/root	Directorio personal del usuario administrador (**Root**). No confundir con /.
/sbin	Binarios de **sistema** esenciales (solo para el administrador, ej: fdisk, reboot).
/tmp	Archivos **temporales**. Generalmente se borra al reiniciar.
/usr	Segunda jerarquía mayor (Unix System Resources). Contiene aplicaciones y librerías de usuario (no críticas para el arranque).
/var	Archivos **variables** que cambian constantemente (logs, bases de datos, colas de impresión, correos).

c. Mencione sistemas de archivos soportados por GNU/Linux.

Linux es extremadamente versátil y soporta docenas de sistemas de archivos. Los más comunes son:

Nativos de Linux:

ext4 (Fourth Extended Filesystem): Es el estándar actual por defecto en la mayoría de las distribuciones. Robusto y fiable.

XFS: Muy usado en servidores por su alto rendimiento con archivos grandes.

Btrfs: Moderno, soporta snapshots y gestión de volúmenes (usado por defecto en Fedora).

Swap: No es para guardar archivos, sino para memoria virtual (intercambio).

De otros sistemas:

FAT16, FAT32, exFAT (Compatibilidad universal).

NTFS (Windows).

HFS+ (macOS antiguo).

En red:

NFS (Network File System).

SMB/CIFS (Samba, para compartir con Windows).

d. ¿Es posible visualizar particiones del tipo FAT y NTFS (que son de Windows) en GNU/Linux?
Sí, es totalmente posible.

Lectura: Linux puede leer particiones FAT y NTFS de forma nativa desde hace muchos años.

Escritura: Actualmente, gracias a controladores como NTFS-3G (o el driver nativo más reciente en kernels nuevos), Linux puede escribir en particiones de Windows sin riesgo de corromper los datos. Esto es muy útil para recuperar información de una PC con Windows roto usando un Live USB de Linux.

7. Particiones:

a. Definición. Tipos de particiones. Ventajas y Desventajas.

Definición: Una partición es una división lógica de una unidad de almacenamiento física (disco duro, SSD, pendrive). El sistema operativo trata cada partición como si fuera un disco independiente, aunque físicamente estén en el mismo dispositivo.

Tipos de particiones (Esquema MBR clásico):

Primaria: Son las divisiones principales del disco. En un esquema MBR (Master Boot Record), solo puede haber un máximo de 4 particiones primarias por disco. Son las únicas desde las que se puede arrancar (bootear) un sistema operativo directamente (aunque GRUB ha flexibilizado esto).

Extendida: Es un tipo especial de partición primaria. Se creó para superar el límite de 4 particiones. Solo puede haber 1 partición extendida por disco y no almacena datos directamente; actúa como un "contenedor".

Lógica: Son las particiones que se crean dentro de la partición extendida. Puedes tener tantas como el sistema permita (teóricamente ilimitadas en Linux, limitadas por letras en Windows).

Ventajas:

Aislamiento: Si el sistema de archivos de una partición se corrompe, no afecta a las otras.

Seguridad y Estabilidad: En Linux, separar directorios como /var o /home evita que un llenado de disco accidental (ej: logs infinitos) colapse todo el sistema operativo (la raíz / sigue teniendo espacio).

Multi-boot: Permite instalar varios sistemas operativos en el mismo disco.

Desventajas:

Rigidez: Redimensionar particiones es riesgoso y lento una vez que tienen datos.

Desperdicio: Si asignas demasiado espacio a una partición que no usas, ese espacio queda "cautivo" y no puede ser usado fácilmente por otra partición que esté llena.

b. ¿Cómo se identifican las particiones en *GNU/Linux*? (Considere discos **IDE**, **SCSI** y **SATA**).
Linux utiliza un esquema de nomenclatura basado en archivos dentro del directorio /dev/.

La nomenclatura:

sd (SCSI Disk): Se usa para discos SATA, SCSI, USB y SSDs modernos.

hd (Hard Disk): Se usaba antiguamente para discos IDE/PATA. (Nota: En los Kernels modernos, incluso los discos IDE suelen ser nombrados como sd por la capa libata, pero académicamente se mantiene la distinción).

La letra (El disco): Indica el orden físico del dispositivo.

/dev/sda: Primer disco físico.

/dev/sdb: Segundo disco físico.

El número (La partición):

1 al 4: Reservados para Particiones Primarias y Extendidas.

5 en adelante: Reservados siempre para Particiones Lógicas.

Ejemplo: /dev/sda5 es la primera partición lógica del primer disco SATA/SCSI.

c. ¿Cuántas particiones son necesarias como mínimo para instalar *GNU/Linux*? Nómbrelas indicando tipo de partición, identificación, tipo de File System y punto de montaje.
Mínimo Absoluto: 1 partición (Raíz /).

(Nota técnica: Si tienes poca RAM, el sistema podría fallar sin Swap, pero instalarse, se instala).

Mínimo Recomendado (Estándar): 2 particiones (Raíz y Swap).

Partición	**Tipo**	**File System**	**Punto de Montaje**	**Descripción**
/dev/sda1	Primaria	ext4	/ (Raíz)	Contiene todo el SO y archivos.
/dev/sda2	Primaria	swap	*(Ninguno)*	Memoria de intercambio (RAM virtual).

d. Dar ejemplos de diversos casos de particionamiento dependiendo del tipo de tarea que se deba realizar en su sistema operativo.
PC de Escritorio (Uso personal):

/ (Raíz): Para el sistema y programas (20-50 GB).

/home: Para documentos del usuario. Ventaja: Si reinstalas Linux, no pierdes tus fotos ni documentos porque están en otra partición.

swap: Memoria de intercambio.

Servidor Web/Base de Datos (Producción):

/: Sistema base.

/var: Donde se guardan los logs, webs y bases de datos. Se separa para que si un log crece descontroladamente o hay un ataque DDoS que llene el disco, no colapse el sistema operativo base.

/boot: Partición pequeña solo para el arranque (seguridad).

/tmp: Separada y montada con opción noexec para evitar que se ejecuten scripts maliciosos desde ahí.

Dual Boot (Windows + Linux):

/dev/sda1: Reservado por el sistema Windows (EFI/Recuperación).

/dev/sda2: C: de Windows (NTFS).

/dev/sda3: / Raíz Linux (ext4).

/dev/sda4: /home o Datos compartidos (NTFS/ext4).

e. ¿Qué tipo de software para particionar existe? Menciónelos y compare.

Software	Tipo	Descripción y Comparación
fdisk	CLI	El clásico universal. Muy potente pero solo soporta tablas de partición MBR (en versiones viejas) y es puro texto. Viene en todos los Linux.
cfdisk	CLI	Similar a fdisk pero con una interfaz de "menú" basado en texto (ncurses) más amigable.
parted	CLI	Más avanzado que fdisk. Soporta discos de más de 2TB (GPT) y permite redimensionar particiones por comando.
GParted	GUI	La herramienta gráfica por excelencia (GNOME Partition Editor). Es muy visual, permite redimensionar arrastrando bloques y gestiona casi cualquier File System.

Software	Tipo	Descripción y Comparación
KDE Partition Manager	GUI	El equivalente a GParted para entornos KDE (Qt).

8. Arranque (*bootstrap*) de un Sistema Operativo:
   a. ¿Qué es el **BIOS**? ¿Qué tarea realiza?
   BIOS (Basic Input/Output System): Es un software de bajo nivel (firmware) almacenado en un chip de memoria no volátil (ROM/Flash) en la placa madre. Es el primer programa que se ejecuta al encender la PC.

   Tarea: Su función principal es inicializar y probar el hardware del sistema (proceso POST - Power-On Self Test) para asegurarse de que la RAM, el teclado y los discos funcionen. Luego, busca un dispositivo de arranque (disco duro, USB, CD) y carga el primer sector de ese disco (MBR) en la memoria para cederle el control.

   b. ¿Qué es **UEFI**? ¿Cuál es su función?
   UEFI (Unified Extensible Firmware Interface): Es el sucesor moderno del BIOS.

   Función: Realiza las mismas tareas de inicialización que el BIOS pero supera sus limitaciones.

   Soporta discos de más de 2 TB (gracias a GPT).

   Ofrece una interfaz gráfica y soporte para mouse antes de cargar el SO.

   Incluye Secure Boot (arranque seguro) para evitar que se cargue software malicioso (rootkits) al inicio.

   Carga los sistemas operativos leyendo archivos .efi directamente de una partición especial (ESP), en lugar de leer código bruto del sector 0 del disco.

   c. ¿Qué es el **MBR**? ¿Qué es el **MBC**?
   MBR (Master Boot Record): Es el primer sector físico (los primeros 512 bytes) de un dispositivo de almacenamiento particionado bajo el esquema clásico. Contiene la tabla de particiones y el código de arranque.

   MBC (Master Boot Code): Es la parte ejecutable del MBR. Ocupa los primeros 446 bytes del MBR. Es el pequeño programa que examina la tabla de particiones para encontrar cuál está marcada como "activa" y carga el sector de arranque de esa partición (Volume Boot Record) para iniciar el sistema operativo.

d. ¿A qué hacen referencia las siglas **GPT**? ¿Qué sustituye? Indique cuál es su formato.
GPT (GUID Partition Table): Tabla de Particiones con Identificador Global Único.

Sustituye a: El esquema MBR. Es parte del estándar UEFI.

Formato/Ventajas:

Utiliza direcciones de 64 bits, permitiendo discos mayores a 2 TB (Zettabytes teóricamente).

Permite hasta 128 particiones primarias por defecto (adiós a las particiones extendidas/lógicas).

Es más robusto: Guarda una copia de la tabla de particiones al final del disco por si la principal se daña.

Usa CRCs (códigos de redundancia cíclica) para detectar corrupción de datos.

e. ¿Cuál es la funcionalidad de un "Gestor de Arranque"? ¿Qué tipos existen? ¿Dónde se instalan? Cite gestores de arranque conocidos.
Funcionalidad: Es el software encargado de cargar el Kernel del sistema operativo en la memoria RAM y pasarle el control. También permite elegir entre varios sistemas operativos instalados (Dual Boot).

Dónde se instalan:

En BIOS/MBR: En el MBR (primera etapa) y en el espacio entre el MBR y la primera partición (segunda etapa).

En UEFI: En la Partición del Sistema EFI (ESP), una partición FAT32 especial.

Tipos y Ejemplos:

GRUB2 (Grand Unified Bootloader): El estándar actual en GNU/Linux. Muy potente y flexible.

LILO (Linux Loader): Antiguo, casi obsoleto, usado en viejos Linux.

Syslinux: Usado comúnmente en Live USBs o arranques por red.

Windows Boot Manager: El gestor por defecto de Microsoft.

f. ¿Cuáles son los pasos que se suceden desde que se prende una computadora hasta que el Sistema Operativo es cargado (proceso de *bootstrap*)?
Power On: La fuente de alimentación envía energía.

BIOS/UEFI: Se ejecuta desde la ROM.

POST: Verifica CPU, RAM y periféricos.

Boot Device: Busca dónde arrancar según el orden configurado (USB, CD, Disco).

Bootloader: El BIOS carga el gestor de arranque (ej. GRUB) desde el disco a la RAM.

Kernel: El gestor de arranque carga el núcleo del SO en memoria.

Init: El Kernel inicia el primer proceso del sistema (PID 1) que cargará el resto de servicios.

g. Analice el proceso de arranque en *GNU/Linux* y describa sus principales pasos.

Carga del Kernel: GRUB carga el archivo del kernel (vmlinuz) y una imagen de disco temporal en RAM llamada initramfs (o initrd).

Initramfs: Contiene drivers mínimos necesarios para montar el disco real (ej: si el disco es ext4 o está cifrado).

Mount Root: El kernel monta el sistema de archivos raíz (/) en modo lectura.

Init (PID 1): El kernel ejecuta /sbin/init (hoy en día generalmente es un enlace a Systemd).

Userspace: Systemd (o SysVinit) comienza a levantar servicios en paralelo (Red, entorno gráfico, audio) según el "Target" o "Runlevel" configurado.

Login: Se presenta la pantalla de inicio de sesión (getty o Display Manager).

h. ¿Cuáles son los pasos que se suceden en el proceso de parada (*shutdown*) de *GNU/Linux*?

El usuario envía la señal de apagado.

El proceso Init/Systemd recibe la orden.

Envía la señal SIGTERM a todos los procesos para pedirles que se cierren y guarden datos.

Tras unos segundos, envía SIGKILL para forzar el cierre de los que no respondieron.

Se desmontan los sistemas de archivos (o se remountan como solo lectura) para asegurar integridad.

Se sincronizan los datos en caché al disco (sync).

El Kernel envía la señal ACPI a la placa madre para cortar la energía.

i. ¿Es posible tener en una PC *GNU/Linux* y otro Sistema Operativo instalado? Justifique.

Sí, es posible.

Justificación: Se logra mediante el particionamiento del disco. Cada sistema operativo reside en particiones separadas (o discos distintos).

El mecanismo: El Gestor de Arranque (como GRUB) detecta los otros sistemas operativos (como Windows) y presenta un menú al inicio ("Chainloading") para que el usuario elija cuál arrancar en esa sesión. Esto se conoce comúnmente como Dual Boot.

9. Archivos y editores:

a. ¿Cómo se identifican los archivos en *GNU/Linux*?

En Linux, los archivos se identifican principalmente por su nombre (que distingue mayúsculas de minúsculas, es decir, Hola.txt y hola.txt son archivos diferentes) y su ruta. Internamente, el sistema operativo identifica cada archivo mediante un número único llamado i-node (nodo índice), que contiene los metadatos del archivo (permisos, dueño, ubicación en disco), pero no el nombre.

Nota sobre extensiones: A diferencia de Windows, en Linux las extensiones (.txt, .exe) son opcionales y meramente informativas para el usuario. Un archivo llamado foto.jpg podría ser un texto plano y el sistema lo tratará como tal según su contenido, no su nombre.

b. Investigue el funcionamiento de los editores **vim, nano** y **mcedit**, y los comandos **cat, more y less**.

Editores de Texto (para crear/modificar archivos):

nano: Es el más sencillo y amigable para principiantes. Los atajos de teclado se muestran abajo (ej: Ctrl+O guardar, Ctrl+X salir).

vim (Vi Improved): Muy potente pero con una curva de aprendizaje alta. Es "modal": tiene un modo para escribir (Insert) y otro para dar órdenes (Normal). Es el estándar en servidores.

mcedit: Editor interno del gestor de archivos "Midnight Commander" (estilo Norton Commander antiguo). Visual y con menús.

Visores de Archivo (solo lectura):

cat: Muestra todo el contenido de un archivo de golpe en la terminal. Útil para archivos cortos.

more: Muestra el contenido página por página. Solo permite avanzar.

less: Una versión mejorada de more. Permite avanzar y retroceder (scrollear) por el archivo. ("Less is more").

c. Cree un archivo llamado “prueba.exe” en su directorio personal usando el **vim**. El mismo debe contener su número de alumno y su nombre.
   Vim prueba.exe
   I
   Ignacio Martinez- legajo=04007/1
   Esc
   :wq

d. Investigue el funcionamiento del comando **file**. Pruébelo con diferentes archivos. ¿Qué diferencia nota?
   El comando file analiza el contenido real (los "números mágicos" o cabeceras binarias) de un archivo para decirte qué es, ignorando la extensión.

e. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con el uso de archivos:

i. ii.	cd	vi. vii.	locate
iii. iv.	mkdir	viii. ix.	ls
v.	rmdir	x.	pwd
	ln tail		cp mv
		xi.	find

cd (Change Directory): Cambia el directorio actual.

mkdir (Make Directory): Crea carpetas nuevas.

rmdir (Remove Directory): Borra directorios solo si están vacíos.

ln (Link): Crea enlaces a archivos (simbólicos ln -s o duros).

tail: Muestra las últimas 10 líneas de un archivo (útil para ver logs).

locate: Busca archivos en todo el disco usando una base de datos indexada (es muy rápido, pero requiere actualización).

ls (List): Lista el contenido de un directorio.

pwd (Print Working Directory): Muestra la ruta absoluta de dónde estás parado.

cp (Copy): Copia archivos o carpetas.

mv (Move): Mueve archivos. También se usa para renombrar.

find: Busca archivos en tiempo real recorriendo los directorios (más lento que locate pero más potente).

10. Indique qué comando es necesario utilizar para realizar cada una de las siguientes acciones. Investigue su funcionamiento y parámetros más importantes:
    a. Cree la carpeta **ISOCSO**
       Mkdir ISOCSO

    b. Acceda a la carpeta
       Cd ISOCSO

    c. Cree dos archivos con los nombres **isocso.txt** e **isocso.csv**
       Touch isocso.txt
       Touch isocso.csv

    d. Liste el contenido del directorio actual
       Ls

    e. Visualizar la ruta donde estoy situado
       Pwd

    f. Busque todos los archivos en los que su nombre contiene la cadena “iso*”
       Find -. -name “iso*” (el punto . indica que busca desde el directorio actual)

    g. Informar la cantidad de espacio libre en disco
       df -h (-h es para que lo muestre en formato de gb o mb)

    h. Verifique los usuarios conectados al sistema
       Who

    i. Editar a el archivo **isocso.txt** e ingresar Nombre y Apellido
       Nano isocso.txt
       Se escribe el nombre y apellido y luego ctrl+o enter, crtl+x

    j. Mostrar en pantalla las últimas líneas de un archivo.
       Tail aisocso.txt

11. Investigue el funcionamiento, parámetros y ubicación (directorio) de los siguientes comandos:

- man
- shutdown
- reboot
- halt
- uname
- dmesg
- lspci
- at
- netstat
- head
- tail

Comando	**Funcionalidad**	**Parámetros comunes**	**Ubicación Típica**
man	Interfaz para leer los manuales de referencia del sistema. Es la "ayuda" principal de Linux.	-k [palabra]: Busca comandos por palabra clave. [sección] [comando]: Abre una sección específica (ej: man 2 write).	/usr/bin/man
shutdown	Apaga o reinicia el sistema de forma segura, notificando a los usuarios y deteniendo procesos.	-h now: Apagar ahora. -r: Reiniciar después de apagar. +m: Programa el apagado para	/sbin/shutdown

Comando	Funcionalidad	Parámetros comunes	Ubicación Típica
		dentro de m minutos.	
reboot	Reinicia el sistema operativo. Es un acceso directo o alias equivalente a shutdown -r.	-f: Fuerza el reinicio (sin detener servicios ordenadamente, peligroso).	/sbin/reboot
halt	Detiene el procesador/hardware. A diferencia de shutdown, es más abrupto y detiene la CPU.	-p: Apaga la energía eléctrica del sistema (poweroff) tras detener la CPU.	/sbin/halt
uname	Imprime información básica sobre el sistema y el hardware.	-a: Muestra toda la información. -r: Muestra la versión del Kernel (muy usado). -m: Muestra la arquitectura (ej: x86_64).	/bin/uname
dmesg	Muestra los mensajes del "Kernel Ring Buffer". Sirve para ver qué pasó durante el arranque y detectar hardware conectado.	-T: Muestra las marcas de tiempo en formato legible (fecha/hora).	/bin/dmesg

Comando	**Funcionalidad**	**Parámetros comunes**	**Ubicación Típica**
		-H: Salida paginada y coloreada.	
lspci	Lista todos los dispositivos conectados al bus PCI (placa de video, red, sonido, etc.).	-v: Modo verborrágico (detallado). -k: Muestra qué driver del kernel está usando cada dispositivo.	/usr/bin/lspci
at	Permite programar un comando para que se ejecute una única vez en un momento futuro específico.	at [hora]: Abre un prompt para escribir los comandos. atq: Lista los trabajos pendientes. atrm: Borra un trabajo.	/usr/bin/at
netstat	Muestra conexiones de red, tablas de enrutamiento y estadísticas de interfaces.	-tuln: Muestra puertos **T**CP/**U**DP en escucha (**L**) numéricamente (**N**).	/bin/netstat

Comando	**Funcionalidad**	**Parámetros comunes**	**Ubicación Típica**
		-p: Muestra qué programa/PID usa el puerto.	
head	Muestra las primeras líneas de un archivo de texto.	-n [num]: Especifica cuántas líneas mostrar (por defecto son 10).	/usr/bin/head
tail	Muestra las últimas líneas de un archivo. Fundamental para leer logs en tiempo real.	-n [num]: Cuántas líneas mostrar. -f: **Follow** (seguir). Muestra nuevas líneas a medida que se escriben en el archivo.	/usr/bin/tail

12. Procesos:

a. ¿Qué es un proceso? ¿A que hacen referencia las siglas PID y PPID? ¿Todos los procesos tienen estos atributos en GNU/Linux? Justifique. Indique qué otros atributos tiene un proceso.

Qué es un proceso? Un proceso es una instancia de un programa en ejecución. Mientras que el programa es el código "muerto" almacenado en el disco (el archivo), el proceso es ese código "vivo" cargado en memoria RAM, utilizando recursos (CPU, E/S) y gestionado por el Kernel.

Significado de las siglas:

PID (Process ID): Identificador de Proceso. Es un número único que el sistema asigna a cada proceso para identificarlo.

PPID (Parent Process ID): Identificador del Proceso Padre. Indica quién creó (o lanzó) a este proceso. En Linux, los procesos se crean "heredando" de otro; si abres Firefox desde la terminal, la terminal (bash) es el padre (PPID) de Firefox (PID).
Sí.

Justificación: El Kernel necesita rastrear absolutamente todo lo que corre. El PID es fundamental para asignarle memoria y tiempo de CPU. El PPID es esencial para mantener la jerarquía; cuando un proceso muere, debe reportar su estado a su padre.

Nota técnica: El único caso especial es el proceso init (o systemd), cuyo PID es 1 y su PPID es 0 (creado por el propio kernel al arrancar).

Otros atributos importantes:

UID (User ID): Usuario propietario del proceso (quién lo lanzó).

GID (Group ID): Grupo propietario.

Estado (State): Indica si está corriendo (R), durmiendo (S), detenido (T) o es un zombie (Z).

Prioridad (Nice): Nivel de prioridad para acceder a la CPU.

TTY: La terminal asociada al proceso (si tiene alguna).

b. Investigue el funcionamiento, parámetros y ubicación (directorio) de los siguientes comandos relacionados a procesos. En caso de que algún comando no venga por defecto en la distribución que utiliza deberá proceder a instalarlo:

i. top
ii. htop
iii. ps
iv. pstree
v. kill
vi. pgrep
vii. pkill
viii. killall
ix. renice
x. xkill
xi. atop
xii. nice

Comando	Funcionalidad	Parámetros Comunes	Ubicación Típica
top	Muestra los procesos en tiempo real, ordenados por consumo de CPU/Memoria. Es el administrador de tareas básico de la terminal.	-u [user]: Filtrar por usuario. k: Matar un proceso (interactivo). d [seg]: Cambiar tiempo de refresco.	/usr/bin/top
htop	Versión mejorada de top. Es interactiva, con colores, permite scrollear y usar el mouse. (Suele requerir instalación).	F6: Ordenar por columnas. F9: Matar procesos. F4: Filtrar lista por nombre.	/usr/bin/htop
ps	Muestra una "foto" (snapshot) estática de los procesos actuales. No se actualiza en tiempo real.	aux: Muestra **todos** los procesos de **todos** los usuarios. -ef: Formato estándar similar a aux.	/bin/ps

Comando	Funcionalidad	Parámetros Comunes	Ubicación Típica
pstree	Muestra los procesos en forma de árbol jerárquico, visualizando claramente quién es el padre de quién.	-p: Muestra los PIDs junto al nombre. -h: Resalta el proceso actual.	/usr/bin/pstree
kill	Envía una señal a un proceso (generalmente para cerrarlo) usando su PID.	-9: SIGKILL (Forzar cierre inmediato). -15: SIGTERM (Cierre ordenado, por defecto). -l: Lista todas las señales posibles.	/bin/kill
pgrep	Busca PIDs basándose en el nombre del proceso.	-l: Muestra el nombre junto al ID. -u [user]: Busca procesos de un usuario específico.	/usr/bin/pgrep
pkill	Envía señales (como kill) pero basándose en el nombre del proceso en lugar del PID.	-9: Forzar cierre. -u [user]: Matar procesos de un usuario.	/usr/bin/pkill
killall	Mata **todos** los procesos que tengan el nombre especificado.	-i: Pregunta antes de matar cada uno (interactivo).	/usr/bin/killall

Comando	Funcionalidad	Parámetros Comunes	Ubicación Típica
		-u [user]: Limita a un usuario.	
renice	Cambia la prioridad de un proceso que **ya** se está ejecutando.	+n / -n: Suma o resta valor al nice. -p [PID]: Especifica el proceso a modificar.	/usr/bin/renice
xkill	Herramienta gráfica. Transforma el cursor en una "calavera/X". Al hacer clic en una ventana, la mata forzosamente.	No suele llevar parámetros comunes, se ejecuta y se hace clic.	/usr/bin/xkill
atop	Monitor de rendimiento avanzado. A diferencia de top, registra actividad histórica (disco, red, CPU) para análisis posterior.	-r: Ver un reporte grabado anteriormente. -d: Muestra actividad de disco detallada.	/usr/bin/atop
nice	Ejecuta un programa iniciándolo con una prioridad modificada desde el principio.	-n [valor]: Define la prioridad (de -20 a 19). Ej: nice -n 10 tar...	/usr/bin/nice

13. Proceso de Arranque *SystemV* (https://github.com/systeminit/si):
    a. Enumere los pasos del proceso de inicio de un sistema GNU/Linux, desde que se prende la PC hasta que se logra obtener el login en el sistema.

    BIOS/UEFI: Carga y ejecuta el gestor de arranque (Bootloader).

    Bootloader (GRUB): Carga el Kernel en memoria y el initramfs.

Kernel: Inicializa el hardware, monta el sistema de archivos raíz (/) y ejecuta el primer programa: init.

Init (SystemV): Lee el archivo /etc/inittab para saber en qué "nivel" (Runlevel) debe arrancar.

RC Scripts: Según el nivel elegido, ejecuta los scripts de inicio ubicados en /etc/rcX.d/ (donde X es el número del nivel). Estos levantan la red, montan discos, inician servidores web, etc.

Getty/Login: Finalmente, init lanza el proceso getty en las terminales (tty) o un Display Manager (gráfica) para pedir usuario y contraseña.

b. Proceso **INIT**. ¿Quién lo ejecuta? ¿Cuál es su objetivo?
¿Quién lo ejecuta?: El Kernel. Es el último paso de la carga del núcleo y se convierte en el primer proceso del espacio de usuario.

Objetivo: Es el "padre de todos los procesos" (siempre tiene PID 1). Su función es inicializar el sistema, mantenerlo en el estado operativo deseado (Runlevel) y adoptar procesos huérfanos.

c. RunLevels. ¿Qué son? ¿Cuál es su objetivo?
¿Qué son?: Son "modos de operación" predefinidos del sistema operativo.

Objetivo: Definir qué conjunto de servicios y procesos deben estar corriendo. Permiten, por ejemplo, arrancar la PC solo para mantenimiento (sin red) o en modo servidor (sin gráficos).

d. ¿A qué hace referencia cada nivel de ejecución según el estándar? ¿Dónde se define qué Runlevel ejecutar al iniciar el sistema operativo? ¿Todas las distribuciones respetan estos estándares?
. Aunque puede variar levemente entre distribuciones antiguas, el estándar (LSB) define:

0: Halt (Apagar el sistema).

1 (o S): Single User Mode (Monousuario/Rescate). Sin red, root directo, para mantenimiento crítico.

2: Multi-usuario sin soporte de red (poco usado hoy).

3: Full Multi-User Mode. Multiusuario completo con red, pero solo consola de texto. (Estándar para servidores).

4: No utilizado / Definible por el usuario.

5: Graphical Mode. Igual al 3, pero inicia el entorno gráfico (X11/Wayland). (Estándar para escritorio).

6: Reboot (Reiniciar el sistema).

¿Dónde se define el default?: En la línea id:X:initdefault: del archivo /etc/inittab.

¿Todas las distribuciones respetan esto?: La mayoría sí seguía este estándar, aunque algunas (como Debian/Ubuntu antiguos) usaban el nivel 2 como gráfico por defecto, difiriendo levemente de Red Hat/CentOS.

e. Archivo /etc/inittab. ¿Cuál es su finalidad? ¿Qué tipo de información se almacena en el? ¿Cuál es la estructura de la información que en él se almacena?
Finalidad: Es el archivo de configuración principal del proceso init. Le dice qué hacer en cada nivel.

Estructura: id:runlevels:action:process

id: Nombre único de la entrada.

runlevels: En qué niveles se aplica (ej: 3, 5).

action: Qué hacer (ej: respawn significa "si muere, reinícialo"; wait significa "espera a que termine").

process: El comando a ejecutar.

f. Suponga que se encuentra en el runlevel <X>. Indique qué comando(s) deberá ejecutar para cambiar al runlevel <Y>. ¿Este cambio es permanente? ¿Por qué?

Comando: telinit [nivel] o init [nivel].

Ejemplo: init 0 apaga la PC; init 6 la reinicia; init 3 mata la interfaz gráfica.

¿Es permanente?: No. Este comando solo cambia el estado de la sesión actual. Si reinicias, el sistema volverá a leer el /etc/inittab y arrancará en el nivel por defecto definido allí.

g. Scripts RC. ¿Cuál es su finalidad? ¿Dónde se almacenan? Cuando un sistema GNU/Linux arranca o se detiene se ejecutan scripts, indique cómo determina qué script ejecutar ante cada acción. ¿Existe un orden para llamarlos? Justifique.
Finalidad: Son scripts de Shell (generalmente bash) encargados de iniciar (start) o detener (stop) un servicio específico (como Apache, SSH, Network).

Almacenamiento: Los scripts reales están en /etc/init.d/.

¿Cómo determina qué ejecutar?: En los directorios /etc/rc0.d/, /etc/rc1.d/, etc., existen enlaces simbólicos (accesos directos) que apuntan a los scripts reales. El nombre del enlace es la clave:

Si empieza con S (Start): Init ejecuta el script con el parámetro start.

Si empieza con K (Kill): Init ejecuta el script con el parámetro stop.

¿Existe un orden?: Sí. Después de la S o K hay un número (ej: S10network, S80httpd). Init los ejecuta en orden numérico ascendente (primero el 10, luego el 80). Esto asegura, por ejemplo, que la red arranque antes que el servidor web que depende de ella.

14. *SystemD* (https://github.com/systemd/systemd):
    a. ¿Qué es *systemd*?
    ¿Qué es systemd? Systemd es un gestor del sistema y de los servicios para Linux. Su función principal es ser el proceso Init (el primero que arranca, con PID 1) y coordinar el resto del arranque.

    Diferencia clave con SystemV: Systemd está diseñado para el paralelismo. En lugar de arrancar servicios uno por uno secuencialmente (como hacía SysV), Systemd lanza muchos servicios al mismo tiempo aprovechando los procesadores modernos, lo que hace que el arranque sea mucho más rápido.
    b. ¿A qué hace referencia el concepto de *Unit* en SystemD?
    Una Unit (Unidad) es el objeto básico que Systemd sabe administrar. Cualquier recurso que Systemd deba controlar se define en un archivo de configuración de "Unidad". Existen varios tipos, que se identifican por su extensión:

    .service: Un servicio o daemon (ej: apache2.service, ssh.service). Es lo más común.

    .socket: Un socket de red o IPC (activa servicios bajo demanda cuando llega tráfico).

    .mount: Un punto de montaje del sistema de archivos.

    .device: Un dispositivo de hardware detectado por el Kernel.
    c. ¿Para qué sirve el comando *systemctl* en SystemD?
    systemctl es la herramienta de línea de comandos principal para controlar Systemd. Es la "navaja suiza" que reemplaza a los viejos comandos service y chkconfig.

    Usos comunes:

    systemctl start [servicio]: Inicia un servicio.

    systemctl stop [servicio]: Lo detiene.

    systemctl status [servicio]: Muestra si está corriendo y sus últimos logs.

systemctl enable [servicio]: Configura el servicio para que arranque automáticamente al encender la PC.

d. ¿A qué hace referencia el concepto de *target* en SystemD?
Un Target es un grupo de Unidades que sirven para sincronizar el arranque y llevar al sistema a un estado específico. Es el reemplazo moderno y flexible de los Runlevels.

Un target básicamente dice: "Para llegar a este estado, necesito que arranquen todos estos servicios".

Equivalencias:

multi-user.target ≈ Runlevel 3 (Modo texto con red).

graphical.target ≈ Runlevel 5 (Modo gráfico).

poweroff.target ≈ Runlevel 0.

e. Ejecutar el comando *pstree*. ¿Qué es lo que se puede observar a partir de la ejecución de este comando?
Al ejecutar pstree, observarás un diagrama de árbol de todos los procesos activos.

Lo que notarás: En la "raíz" o copa del árbol (la izquierda de la pantalla), el primer proceso ya no es un init genérico, sino explícitamente systemd.

Conclusión: Verás gráficamente cómo todos los demás procesos del sistema (navegadores, terminales, servicios de red) son "hijos" o ramas que nacen directa o indirectamente de Systemd, confirmando su rol como el padre de todo el espacio de usuario (PID 1).

15. Usuarios:

a. ¿Qué archivos son utilizados en un sistema GNU/Linux para guardar la información de los usuarios?
Son archivos de texto plano ubicados en /etc. Los tres principales son:

/etc/passwd: Contiene la información pública de las cuentas (Nombre, UID, GID, directorio home, shell). Es legible por todos los usuarios.

/etc/shadow: Contiene las contraseñas encriptadas (hash) y las políticas de expiración. Solo es legible por el usuario root (por seguridad).

/etc/group: Contiene la información de los grupos y qué usuarios pertenecen a ellos.

b. ¿A qué hacen referencia las siglas *UID* y *GID*? ¿Pueden coexistir UIDs iguales en un sistema GNU/Linux? Justifique.
UID (User IDentifier): Es el número único que identifica al usuario ante el Kernel. El sistema operativo no sabe de nombres ("juan"), solo de números.

GID (Group IDentifier): Es el número que identifica al grupo principal del usuario.

¿Pueden coexistir UIDs iguales?

Técnicamente SÍ: El sistema lo permite. Puedes editar manualmente /etc/passwd y ponerle el UID 1001 a "pedro" y a "maria".

Consecuencia: Para el Kernel, son la misma persona. Si "pedro" crea un archivo, "maria" será dueña de ese archivo automáticamente y tendrá todos sus permisos. Generalmente se considera una mala práctica o un error de administración, salvo casos muy específicos de aliasing

c. ¿Qué es el usuario root? ¿Puede existir más de un usuario con este perfil en GNU/Linux? ¿Cuál es la *UID* de *root*?

¿Qué es?: Es el superusuario o administrador del sistema. Tiene acceso total a todos los comandos y archivos, ignorando cualquier restricción de permisos.

UID: Su UID es siempre 0.

¿Puede existir más de uno?

Por nombre: Solo hay un usuario llamado "root" por defecto.

Por privilegios: Sí. Cualquier usuario al que le asignes manualmente el UID 0 en el archivo /etc/passwd se convertirá instantáneamente en un superusuario con los mismos poderes que root, aunque se llame "admin" o "pepe". (Nuevamente, no es recomendado por seguridad).

d. Agregue un nuevo usuario llamado *isocso* a su instalación de GNU/Linux, especifique que su home sea creada en /home/*isocso*, y hágalo miembro del grupo *informatica* (si no existe, deberá crearlo). Luego, sin iniciar sesión como este usuario cree un archivo en su home personal que le pertenezca. Luego de todo esto, borre el usuario y verifique que no queden registros de él en los archivos de información de los usuarios y grupos.

Sudo groupadd informática
Sudo useradd -d /home/isocso -m -g informatica isocso
Sudo -u isocso touch /home/isocso/archive_propio.txt
Sudo userdel -r isocso
Grep isocso /etc/passwd
Grep informatica /etc/group
Ls /home/isocso

e. Investigue la funcionalidad y parámetros de los siguientes comandos:

Comando	**Descripción**
useradd	Comando de bajo nivel para crear usuarios. Si no le pasas parámetros (como -m), a veces crea el usuario sin home ni configuración básica.
adduser	(En distros como Debian/Ubuntu) Es un script más amigable e interactivo que usa useradd por detrás. Te hace preguntas (nombre completo, contraseña, etc.) y configura todo automáticamente.
usermod	Modifica una cuenta existente (cambiarla de grupo, bloquearla, cambiar su

Comando	Descripción
	home, etc.). Ej: usermod -aG sudo usuario.
userdel	Elimina una cuenta de usuario.
su	(**S**witch **U**ser). Permite cambiar la sesión actual a la de otro usuario. Si se usa solo, intenta cambiar a root.
groupadd	Crea un nuevo grupo de trabajo.
who	Muestra quiénes están logueados actualmente en el sistema.
groupdel	Elimina un grupo existente.

Comando	Descripción
passwd	Asigna o cambia la contraseña de un usuario. (Ej: passwd cambia la tuya, sudo passwd usuario cambia la de otro).

16. FileSystem y permisos:
    a. ¿Cómo son definidos los permisos sobre archivos en un sistema GNU/Linux?

Las Entidades (¿Quién?):

u (User/Owner): El dueño del archivo (generalmente quien lo creó).

g (Group): El grupo dueño del archivo. Cualquier usuario que pertenezca a este grupo comparte estos permisos.

o (Others): "El resto del mundo". Cualquier usuario que no sea el dueño ni pertenezca al grupo.

Los Permisos (¿Qué?):

r (Read):

Archivo: Ver su contenido.

Directorio: Listar los archivos dentro (ls).

w (Write):

Archivo: Modificar o borrar el contenido del archivo.

Directorio: Crear, borrar o renombrar archivos dentro de ese directorio.

x (Execute):

Archivo: Ejecutarlo como un programa/script.

Directorio: Entrar en él (cd). Sin permiso x, no puedes acceder a la carpeta aunque tengas permiso r para ver sus nombres.

b. Investigue la funcionalidad y parámetros de los siguientes comandos relacionados con los permisos en GNU/Linux:
   i. chmod
   ii. chown
   iii. chgrp

Comando	Significado	Función	Ejemplo
chmod	**Ch**ange **Mod**e	Cambia los permisos de lectura/escritura/ejecución.	chmod u+x script.sh (Da ejecución al dueño).
chown	**Ch**ange **Own**er	Cambia el usuario dueño del archivo (y opcionalmente el grupo).	chown ignacio archivo.txt (Hace a "ignacio" el dueño).
chgrp	**Ch**ange **Group**	Cambia solamente el grupo dueño del archivo.	chgrp alumnos tp.pdf (Asigna el grupo "alumnos").

c. Al utilizar el comando chmod generalmente se utiliza una notación octal asociada para definir permisos. ¿Qué significa esto? ¿A qué hace referencia cada valor?
La notación octal es una forma matemática de representar los permisos combinando valores numéricos. Cada permiso tiene un valor:

r (Leer) = 4

w (Escribir) = 2

x (Ejecutar) = 1

- (Nada) = 0

Se suman los valores para cada entidad (Usuario, Grupo, Otros) formando un número de 3 dígitos.

Ejemplo Clásico: chmod 755 archivo

7 (Dueño): 4+2+1 = rwx (Lectura, escritura y ejecución).

5 (Grupo): 4+0+1 = r-x (Lectura y ejecución).

5 (Otros): 4+0+1 = r-x (Lectura y ejecución).

d. ¿Existe la posibilidad de que algún usuario del sistema pueda acceder a determinado archivo para el cual no posee permisos? Indiquelo y realice las pruebas correspondientes.
Usuario normal: NO. El Kernel impide estrictamente el acceso. Si intentas leer un archivo sin permiso r, recibirás "Permission denied".

Excepción: El usuario Root ignora todos los permisos. Puede leer y escribir cualquier cosa, incluso si los permisos son 000.

e. Explique los conceptos de “full path name” (path absoluto) y “relative path name” (path relativo). De ejemplos claros de cada uno de ellos.
Full Path Name (Absoluto): Es la ruta completa e inequívoca desde la raíz (/). No importa en qué carpeta estés parado, esta ruta siempre funciona.

Ejemplo: /home/ignacio/Descargas/foto.jpg

Relative Path Name (Relativo): Es la ruta buscada desde donde estás parado actualmente. No empieza con /.

Ejemplo: Si estás en /home/ignacio, escribes: Descargas/foto.jpg.

Símbolos clave: . (aquí mismo) y .. (el directorio padre/atrás).

f. ¿Con qué comando puede determinar en qué directorio se encuentra actualmente? ¿Existe alguna forma de ingresar a su directorio personal sin necesidad de escribir todo el path completo? ¿Podría utilizar la misma idea para acceder a otros directorios? ¿Cómo? Explique con un ejemplo.

¿Dónde estoy?: Comando pwd (Print Working Directory).

Ir a mi home sin escribir todo:

Solo escribe cd y presiona Enter.

O usa el símbolo virgulilla: cd ~.

Acceder al home de otros:

Usa la virgulilla seguida del nombre del usuario: cd ~profesor/documentos (te llevará a /home/profesor/documentos).

Comando	Función
mount	"Monta" un sistema de archivos (disco, USB, ISO) en un directorio del árbol principal para hacerlo accesible. Ej: mount /dev/sdb1 /mnt/usb.
umount	Desmonta el dispositivo para poder retirarlo de forma segura (asegura que no haya datos pendientes de escritura).
df	(**D**isk **F**ree). Muestra el espacio libre y ocupado de las particiones montadas. Útil con -h (human readable).
du	(**D**isk **U**sage). Calcula el tamaño real que ocupa un directorio y sus subcarpetas. Ej: du -sh Fotos/.
mkfs	(**M**ake **F**ile**S**ystem). Formatea una partición. Ej: mkfs.ext4 /dev/sdb1. **Cuidado:** Borra todo.
fdisk	Herramienta para crear y borrar particiones en un disco (manipula la tabla de particiones).
write	Envía mensajes de texto en tiempo real a la terminal de otro usuario conectado. (Ej: write pepe y escribes el mensaje).
losetup	Configura "Loop devices". Permite montar un archivo normal (como una imagen .iso o .img) como si fuera un disco físico real /dev/loop0.
stat	Muestra información detallada del **inode** de un archivo: fechas exactas de acceso/modificación, dueño, permisos en octal y tamaño en bloques.

17. Procesos:
    a. ¿Qué significa que un proceso se está ejecutando en Background? ¿Y en Foreground?
    Foreground (Primer Plano): Es el modo estándar. El proceso toma el control de la terminal y bloquea la entrada de nuevos comandos hasta que finaliza. El usuario interactúa directamente con él.

Background (Segundo Plano): El proceso se ejecuta "detrás de escena", liberando la terminal inmediatamente para que el usuario pueda seguir trabajando en otras cosas mientras el proceso continúa su tarea.

b. ¿Cómo puedo hacer para ejecutar un proceso en Background? ¿Como puedo hacer para pasar un proceso de background a foreground y viceversa?
Para ejecutar un proceso de background seria sleep 60 &
Presionar Ctrl + Z (Detiene/Pausa el proceso).

Escribir el comando bg (Lo reanuda en segundo plano).

Traer de Background a Foreground:

Comando fg (Si hay varios, usar fg %1, fg %2).

c. Pipe (**|**). ¿Cuál es su finalidad? Cite ejemplos de su utilización.
Permite conectar la salida estándar (stdout) de un comando con la entrada estándar (stdin) del siguiente.

Ejemplo 1: ls -l | more (Lista archivos y los pasa al paginador para verlos poco a poco).

Ejemplo 2: ps aux | grep systemd (Busca el texto "systemd" dentro del listado de procesos).

d. Redirección. ¿Qué tipo de redirecciones existen? ¿Cuál es su finalidad? Cite ejemplos de utilización. gar de pantalla/teclado).

> (Salida): Guarda la salida en un archivo (sobrescribe). Ej: ls > lista.txt.

>> (Salida): Agrega la salida al final de un archivo (no borra). Ej: echo "Fin" >> log.txt.

< (Entrada): Toma el contenido de un archivo como entrada para el comando. Ej: mysql < base_datos.sql.

18. Otros comandos de Linux (Indique funcionalidad y parámetros):

a. ¿A qué hace referencia el concepto de empaquetar archivos en GNU/Linux?
empaquetar (Archiving) significa agrupar múltiples archivos y directorios en un único archivo contenedor (comúnmente llamado "tarball"), sin necesariamente comprimirlos (sin reducir su tamaño).

El objetivo es facilitar el transporte o backup, manteniendo los permisos, fechas y estructura de directorios originales.

La herramienta estándar para esto es tar.

b. Seleccione 4 archivos dentro de algún directorio al que tenga permiso y sume el tamaño de cada uno de estos archivos. Cree un archivo empaquetado conteniendo estos 4 archivos y compare los tamaños de los mismos. ¿Qué característica nota?

Acción: Si sumas el tamaño de 4 archivos de 1MB cada uno y creas un paquete .tar, el archivo resultante pesará 4MB (o un poquito más por los metadatos/cabeceras).

Característica que se nota: El tamaño NO disminuye. A diferencia de un .zip en Windows, el comando tar por sí solo no comprime, solo "junta" los archivos en una caja.

c. ¿Qué acciones debe llevar a cabo para comprimir 4 archivos en uno solo? Indique la secuencia de comandos ejecutados.

Empaquetar: Usar tar para crear un solo archivo .tar.

tar -cvf archivo.tar file1 file2 file3 file4

Comprimir: Usar una herramienta de compresión (como gzip) sobre ese archivo.

gzip archivo.tar

Resultado final: archivo.tar.gz

d. ¿Pueden comprimirse un conjunto de archivos utilizando un único comando?

Sí. Las versiones modernas de tar incluyen "banderas" (flags) que llaman automáticamente al compresor, haciendo todo en un solo paso.

Comando: tar -czvf paquete.tar.gz archivo1 archivo2...

-c: Crear.

-z: Usar Gzip (compresión).

-v: Verbose (mostrar en pantalla lo que hace).

-f: File (nombre del archivo).

e. Investigue la funcionalidad de los siguientes comandos:

i. tar
ii. grep
iii. gzip
iv. zgrep
v. wc

Comando	Funcionalidad
tar	(**T**ape **AR**chiver). Herramienta principal para empaquetar archivos. Soporta compresión mediante parámetros extra (-z para gzip, -j para bzip2).
grep	(**G**lobal **R**egular **E**xpression **P**rint). Busca cadenas de texto o patrones dentro de archivos o salidas de comandos.
gzip	Herramienta de compresión estándar (algoritmo Lempel-Ziv). Reduce el tamaño de los archivos. Generalmente reemplaza el archivo original por la versión .gz.
zgrep	Igual que grep, pero permite buscar texto **dentro de archivos comprimidos** (.gz) sin necesidad de descomprimirlos primero.
wc	(**W**ord **C**ount). Cuenta saltos de línea, palabras y bytes de un archivo. • wc -l: Cuenta solo líneas (muy útil para contar resultados en pipes).

19. Indique qué acción realiza cada uno de los comandos indicados a continuación considerando su orden. Suponga que se ejecutan desde un usuario que no es root ni pertenece al grupo de root. (Asuma que se encuentra posicionado en el directorio de trabajo del usuario con el que se logueó). En caso de no poder ejecutarse el comando, indique la razón:

```
l s -l > prueba  ps >
PRUEBA  chmod 710
prueba  chown
root:root PRUEBA
chmod 777 PRUEBA
chmod 700 /etc/passwd
passwd root  rm PRUEBA
```

```
man /etc/shadow  find
/ -name * .conf
usermod root -d /home/ newroot -L
cd / root  rm *  cd / etc  cp *
/home -R  shutdown
```

20. Indique qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:
    a. Terminar el proceso con *PID* 23.
       Kill 23
    b. Terminar el proceso llamado *init* o *systemd*. ¿Qué resultados obtuvo?
       Kill 1
    c. Buscar todos los archivos de usuarios en los que su nombre contiene la cadena ".conf"
       Find /home -name "*.conf*"
    d. Guardar una lista de procesos en ejecución el archivo **/home/<su nombre de usuario>/procesos**
       Ps aux >/home/nombre/procesos
    e. Cambiar los permisos del archivo **/home/<su nombre de usuario>/xxxx** a:
       i. Usuario: Lectura, escritura, ejecución
       ii. Grupo: Lectura, ejecución
       iii. Otros: ejecución
       Chmod 751 /home/<su nombre de usuario>/xxxx
    f. Cambiar los permisos del archivo **/home/<su nombre de usuario>/yyyy** a:
       i. Usuario: Lectura, escritura. ii. Grupo: Lectura, ejecución
       iii. Otros: Ninguno
    g. Borrar todos los archivos del directorio **/tmp**
       Rm -rf /tmp/*
    h. Cambiar el propietario del archivo **/opt/isodata** al usuario **isocso**
       Sudo Chown isocso /opt/isodata
    i. Guardar en el archivo **/home/<su nombre de usuario>/donde** el directorio donde me encuentro en este momento, en caso de que el archivo exista no se debe eliminar su contenido anterior.
       Mv **/home/<su nombre de usuario>/donde pwd**

21. Indique qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:
    a. Ingrese al sistema como usuario "root"
       Su -
    b. Cree un usuario. Elija como nombre, por convención, la primera letra de su nombre seguida de su apellido. Asígnele una contraseña de acceso.

Useradd -m imartinez
Passwd imartinez

c. ¿Qué archivos fueron modificados luego de crear el usuario y qué directorios se crearon?
Se modificaron los archivos /etc/passwd y /etc/shadow y /etc/group

d. Crear un directorio en /tmp llamado *miCursada*
Mkdir /tmp/miCursada

e. Copiar todos los archivos de /var/log al directorio antes creado.
Cp -r /var/log/* /tmp/miCursada/

f. Para el directorio antes creado (y los archivos y subdirectorios contenidos en él) cambiar el propietario y grupo al usuario creado y grupo users.
Chown -R imartinez:users /tmp(miCUrsada

g. Agregue permiso total al dueño, de escritura al grupo y escritura y ejecución a todos los demás usuarios para todos los archivos dentro de un directorio en forma recursiva.
Chmod -r 723 /tmp/micursada

h. Acceda a otra terminal para loguearse con el usuario antes creado.
Una vez logueado con el usuario antes creado, averigüe cuál es el nombre de su terminal.
tty

i. Verifique la cantidad de procesos activos que hay en el sistema.
Ps aux |wc -l

j. Verifiqué la cantidad de usuarios conectados al sistema.
Who |wc -l

k. Vuelva a la terminal del usuario root y envíele un mensaje al usuario anteriormente creado enviándole que el sistema va a ser apagado.
Write irmartinez

l. Apague el sistema.
Shutdown now

22. Indique qué comando sería necesario ejecutar para realizar cada una de las siguientes acciones:

n. Cree un directorio cuyo nombre sea su número de legajo e ingrese a él.

o. Cree un archivo utilizando el editor de textos *vi*, e introduzca su información personal: Nombre, Apellido, Número de alumno y dirección de correo electrónico. El archivo debe llamarse "LEAME".

p. Cambie los permisos del archivo LEAME, de manera que se puedan ver reflejados los siguientes permisos:
   - Dueño: ningún permiso
   - Grupo: permiso de ejecución
   - Otros: todos los permisos

q. Vaya al directorio /etc y verifique su contenido. Cree un archivo dentro de su directorio personal cuyo nombre sea **leame** donde el contenido del mismo sea el listado de todos los archivos y directorios contenidos en /etc. ¿Cuál es la razón por la cuál puede crear este archivo si ya existe un archivo llamado "LEAME" en este directorio?

r. ¿Qué comando utilizaría y de qué manera si tuviera que localizar un archivo dentro del filesystem? ¿Y si tuviera que localizar varios archivos con características similares? Explique el concepto teórico y ejemplifique.

s. Utilizando los conceptos aprendidos en el punto anterior, busque todos los archivos cuya extensión sea .so y almacene el resultado de esta búsqueda en un archivo dentro del directorio creado en el primer inciso. El archivo deberá llamarse *ejercicioF*.

23. Indique qué acción realiza cada uno de los comandos indicados a continuación considerando su orden. Suponga que se ejecutan desde un usuario que no es root ni pertenece al grupo de root. (Asuma que se encuentra posicionado en el directorio de trabajo del usuario con el que se logueó). En caso de no poder ejecutarse el comando indique la razón:

```
01.  mkdir iso
02.  cd . / iso; ps > f0
03.  ls > f1
04.  cd /
05.  echo $HOME
06.  ls -l $> $HOME/ iso/ls
07.  cd $HOME; mkdir f2
08.  ls -ld f2
09.  chmod 341 f2
10.  touch dir
11.  cd f2
12.  cd ~/iso
13.  pwd > f3
14.  ps | grep 'ps' | wc -l >> ../f2/f3
15.  chmod 700 ../f2 ; cd ..
16.  find . -name etc/passwd
17.  find / -name etc/passwd
18.  mkdir ejercicio5
19.  . . . . . . . . . . . . . . . . . . . . 20.
     . . . . . . . . . . . . . . . . . . . .
```

a. Inicie 2 sesiones utilizando su nombre de usuario y contraseña. En una sesión vaya siguiendo paso a paso las órdenes que se encuentran escritas en el cuadro superior. En la otra sesión, cree utilizando algún editor de textos un archivo que se llame “explicacion_de_ejercicio" dentro del directorio creado en el ejercicio 22 y, para cada una de los comandos que ejecute en la otra sesión, realice una breve explicación de los resultados obtenidos.

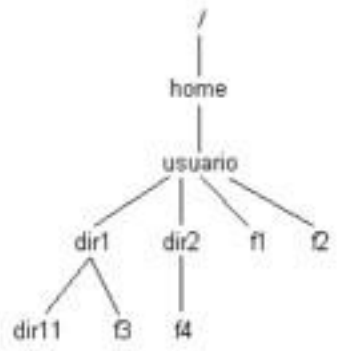
b. Complete los comandos 19 y 20, de manera tal que realicen la siguiente acción:

   19: Copiar el directorio iso y todo su contenido al directorio creado en 24.a

   20: Copiar el resto de los archivos y directorios que se crearon en este ejercicio al directorio creado en el ejercicio 24.a

c. Ejecute las órdenes 19 y 20 y coméntelas en el archivo creado en el inciso a).

24. Cree una estructura desde el directorio /home que incluya varios directorios, subdirectorios y archivos, según el esquema siguiente.



Asuma que “usuario” indica cuál es su nombre de usuario. Además deberá tener en cuenta que dirX hace referencia a directorios y fX hace referencia a archivos. Utilizando la estructura de directorios anteriormente creada, indique qué comandos son necesarios para realizar las siguientes acciones:

a. Mueva el archivo "f3” al directorio de trabajo /home/usuario.
b. Copie el archivo "f4” en el directorio "dir11".
c. Haga los mismo que en el inciso anterior pero el archivo de destino, se debe llamar "f7".
d. Cree el directorio copia dentro del directorio usuario y copie en él, el contenido de "dir1".
e. Renombre el archivo "f1" por el nombre *archivo* y vea los permisos del mismo.
f. Cambie los permisos del archivo llamado *archivo* de manera de reflejar lo siguiente:
    - Usuario: Permisos de lectura y escritura
    - Grupo: Permisos de ejecución
    - Otros: Todos los permisos
g. Renombre los archivos "f3” y “f4" de manera que se llamen "f3.exe” y “f4.exe” respectivamente.
h. Utilizando un único comando cambie los permisos de los dos archivos renombrados en el inciso anterior, de manera de reflejar lo siguiente:
    - Usuario: Ningún permiso

- ➢ Grupo: Permisos de escritura
- ➢ Otros: Permisos de escritura y ejecución

25. Indique qué comando/s es necesario para realizar cada una de las acciones de la siguiente secuencia de pasos (considerando su orden de aparición):
    a. Cree un directorio llamado *logs* en el directorio /tmp.
    b. Copie todo el contenido del directorio /var/log en el directorio creado en el punto anterior.
    c. Empaquete el directorio creado en a), el archivo resultante se debe llamar "misLogs.tar".
    d. Empaquete y comprima el directorio creado en a), el archivo resultante se debe llamar "misLogs.tar.gz".
    e. Copie los archivos creados en c) y d) al directorio de trabajo de su usuario.
    f. Elimine el directorio creado en a), *logs*.
    g. Desempaquete los archivos creados en c y d en 2 directorios diferentes.

de 10